

M3C5 (Checkpoint 5)

1- ¿Qué es un condicional?

Un condicional es una estructura de control en programación que permite que el flujo del programa siga diferentes caminos dependiendo de si una condición se cumple o no. En otras palabras, es como tomar decisiones dentro del código: si se cumple una cierta condición, el programa ejecuta el código; si no se cumple, puede ejecutar otras líneas de código diferente o no hacer nada.

Pensando en un condicional como una pregunta que se le hace al programa: "¿Es verdadero esto?". Dependiendo de la respuesta, el programa hace una cosa u otra.

En Python, los condicionales se manejan principalmente con las palabras clave `if`, `elif` y `else`:

1. `if`: Es la palabra clave que inicia un condicional. Se utiliza para verificar si una condición es verdadera.
2. `elif` (else if): Es una extensión del condicional `if` que permite verificar múltiples condiciones. Solo se ejecuta si la condición del `if` anterior es falsa.
3. `else`: Se usa para ejecutar un bloque de código cuando todas las condiciones anteriores son falsas.

En caso de querer escribir un programa que determine si una persona es mayor de edad.

```
'''
edad = 18

if edad >= 18:
    print("puedes beber.")
else:
    print("puedes beber pero no alcohol.")
```

Salida: Puedes beber.

En este ejemplo:

- El condicional `if` verifica si la edad es mayor o igual a 18.
- Si la condición es verdadera (como en este caso), se ejecuta el primer bloque de código, y se imprime "Puedes beber".
- Si la condición fuera falsa, se ejecutaría el bloque de código en `else`, imprimiendo "Puedes beber pero no alcohol".

Las condiciones son expresiones que se evalúan como `True` (verdadero) o `False` (falso). Aquí hay algunos ejemplos de expresiones condicionales:

- Comparaciones:
 - ==: Igual a.
 - !=: No igual a.
 - >: Mayor que.
 - <: Menor que.
 - >=: Mayor o igual que.
 - <=: Menor o igual que.

Ejemplo:

```
'''
a = 10
b = 5
print(a > b) # True
'''
```

- Operadores lógicos:
 - and: Verifica si ambas condiciones son verdaderas.
 - or: Verifica si al menos una de las condiciones es verdadera.
 - not: Invierte el valor de una condición.

Ejemplo:

```
'''
a = 10
b = 5
c = 3

if a > b and b > c:
    print("Ambas condiciones son verdaderas")
'''
```

Salida: Ambas condiciones son verdaderas

Si queremos clasificar la edad de una persona en diferentes categorías, podemos utilizar los condicionales vistos hasta ahora:

```
'''
edad = 15

if edad < 13:
    print("Eres un niño.")
elif 13 <= edad < 18:
    print("Eres un adolescente.")
elif 18 <= edad < 65:
    print("Eres un adulto.")
else:
    print("Eres un adulto mayor.")
'''
```

Salida: Eres un adolescente.

En el ejemplo anterior se ha seguido el siguiente orden de decisiones:

1. Primera condición (`if edad < 13`): El programa verifica si la edad es menor a 13. Si es verdadera, ejecuta el código asociado, pero en este caso es falsa, por lo que sigue adelante.
2. Segunda condición (`elif 13 <= edad < 18`): El programa verifica si la edad está entre 13 y 18. Como es verdadera, se ejecuta el código asociado y se imprime "Eres un adolescente".
3. Siguiente (`elif 18 <= edad < 65`): Este bloque no se ejecuta porque ya se ejecutó el bloque anterior.
4. Último (`else`): Este bloque tampoco se ejecuta, ya que una de las condiciones anteriores ya fue verdadera.

A veces, se pueden necesitar condicionales dentro de otros condicionales. Esto se llama anidar condicionales y puede ser útil cuando tienes que tomar decisiones más complejas.

```
...
edad = 25
trabaja = True

if edad >= 18:
    if trabaja:
        print("Eres un adulto trabajador.")
    else:
        print("Eres un adulto sin trabajo.")
else:
    print("Eres menor de edad.")
...
```

Salida: Eres un adulto trabajador.

En este ejemplo, se verifica primero si la persona es mayor de edad. Si es cierto, se realiza una segunda verificación para ver si la persona trabaja.

Los condicionales son una herramienta esencial en Python que nos aportará en muchos aspectos a lo largo del desarrollo en programación:

- Toman decisiones: Los condicionales permiten que un programa decida qué hacer en diferentes situaciones, lo que hace que el programa sea más dinámico y adaptable.
- Controlan el flujo del programa: Sin condicionales, el código se ejecutaría de manera lineal, sin poder reaccionar a diferentes situaciones.
- Facilitan la personalización: Permiten que el mismo código funcione de manera diferente en función de las entradas del usuario o de los datos que recibe.

...

2- ¿Cuáles son los diferentes tipos de bucles en Python? ¿Por qué son útiles?

Un bucle es una estructura que permite repetir un bloque de código varias veces. En lugar de escribir el mismo código una y otra vez, los bucles nos permiten ejecutarlo repetidamente, lo que hace que nuestro programa sea más eficiente.

Los bucles son extremadamente útiles cuando necesitamos realizar una tarea repetitiva, como procesar elementos en una lista, leer líneas de un archivo, o generar números secuenciales. En Python, existen dos tipos principales de bucles:

1. Bucle `for`
2. Bucle `while`

1. Bucle `for`:

El bucle `for` se utiliza cuando se sabe de antemano cuántas veces se debe repetir un bloque de código. Este tipo de bucle es ideal para iterar sobre una secuencia de elementos, como una lista, una string, o un rango de números.

Escritura del Bucle `for`:

```
'''
```

```
for variable in secuencia:
```

```
'''
```

- `variable`: Es el nombre de una variable que toma el valor de cada elemento en la secuencia.
- `secuencia`: Es la colección de elementos (como una lista, una cadena, o un rango) sobre la cual se itera.

Ejemplo:

```
'''
```

```
numeros = [1, 2, 3, 4, 5]
```

```
for numero in numeros:
```

```
    print(numero)
```

```
'''
```

Salida:

```
1
2
3
4
5
```

En este ejemplo, el bucle `for` recorre cada elemento de la lista `numeros` y lo imprime. La variable `numero` toma el valor de cada elemento de la lista en cada iteración.

Iterando sobre una string:

```
'''
```

```
palabra = "Python"
```

```
for letra in palabra:
```

```
    print(letra)
```

```
'''
```

Salida:

P
y
t
h
o
n

En este ejemplo, el bucle `for` recorre cada carácter de la cadena de texto `palabra` y lo imprime.

Puede haber bucles `for` dentro de otros bucles `for`, a lo que se le llama bucles anidados. Por ejemplo:

```
matriz = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
for fila in matriz:  
    for elemento in fila:  
        print(elemento)  
    print()
```

Salida:

```
123  
456  
789
```

2. Bucle `while`:

El bucle `while` se utiliza cuando no se sabe cuántas veces se repetirá un bloque de código. En lugar de iterar sobre una secuencia, el bucle `while` sigue ejecutándose mientras una condición sea verdadera.

Escritura del Bucle `while`:

```
...
```

```
while condición:  
    # Código a ejecutar mientras la condición sea verdadera
```

- **condición:** Es una expresión que se evalúa como `True` o `False`. Mientras la condición sea `True`, el bucle continuará ejecutándose. Si la condición se evalúa como `False`, el bucle se detiene.

Ejemplo:

```
'''
contador = 0

while contador < 5:
    print("Contador:", contador)
    contador += 1
```

Salida:

```
makefile
'''
Contador: 0
Contador: 1
Contador: 2
Contador: 3
Contador: 4
```

En este ejemplo, el bucle `while` sigue ejecutándose mientras el valor de `contador` sea menor que 5. En cada iteración, el valor de `contador` se incrementa en 1.

Un error a tener en cuenta en los bucles `while` es crear un bucle infinito, donde la condición nunca se vuelve falsa. Esto hará que el bucle se ejecute indefinidamente.

```
'''
while True:
    print("Este es un bucle infinito")
'''
```

Este bucle seguirá ejecutándose hasta que se interrumpa manualmente porque la condición `True` siempre es verdadera.

Ejemplo con Entrada de Usuario:

Los bucles `while` son útiles cuando trabajas con entradas de usuarios, donde no sabes cuántas veces necesitarás ejecutar el bucle.

```
'''
respuesta = ""

while respuesta != "salir":
    respuesta = input("Escribe 'salir' para terminar el programa: ")
    print("Escribiste:", respuesta)
```

Salida:

```
Escribe 'salir' para terminar el programa: hola
Escribiste: hola
Escribe 'salir' para terminar el programa: adiós
Escribiste: adiós
Escribe 'salir' para terminar el programa: salir
```

Escribiste: salir

El bucle continúa hasta que el usuario escribe "salir".

Los bucles son útiles en diversos aspectos como:

1. Automatizan tareas repetitivas: En lugar de escribir el mismo código varias veces, puedes usar un bucle para repetir una tarea.
2. Procesan datos: Los bucles son fundamentales para procesar datos, como recorrer listas, archivos, o resultados de consultas.
3. Interactúan con el usuario: Los bucles permiten que los programas continúen ejecutándose hasta que el usuario decida detenerlos, como en los menús de aplicaciones.
4. Facilitan cálculos complejos: En matemáticas y simulaciones, los bucles permiten realizar cálculos repetitivos de manera eficiente.

Supongamos que estás creando una aplicación que suma todos los números de una lista. En lugar de sumarlos manualmente, puedes usar un bucle `for`. Por ejemplo:

```
...
numeros = [1, 2, 3, 4, 5]
suma = 0

for numero in numeros:
    suma += numero

print("La suma de los números es:", suma)
```

Salida: La suma de los números es: 15

Se resumen, a continuación algunas diferencias de `for` y `while`:

- `for`: Se usa cuando sabes cuántas veces se repetirá el bucle o cuando quieres iterar sobre una secuencia.
- `while`: Se usa cuando no sabes exactamente cuántas veces se repetirá el bucle, pero tienes una condición que controla cuándo debe detenerse.

3- ¿Qué es una comprensión de listas en Python?

Una comprensión de listas es una forma precisa y eficiente de crear listas en Python. En lugar de utilizar bucles tradicionales y añadir elementos a una lista uno por uno, las comprensiones de listas permiten construir una nueva lista aplicando una expresión a cada elemento de una secuencia o una colección de datos.

En otras palabras, las comprensiones de listas combinan la lógica de un bucle `for`, junto con un condicional `if`, para generar una nueva lista en una sola línea de código. En resumen, nos serán útiles por tres motivos principales:

- Escritura Concisa: Las comprensiones de listas permiten crear listas en una sola línea, lo que hace que el código sea más corto y más legible.
- Rendimiento: Son generalmente más rápidas que los bucles tradicionales para crear listas.

- **Expresividad:** Facilitan la escritura de código que es fácil de entender, especialmente cuando se trabaja con operaciones simples de listas.

La Escritura básica de una comprensión de listas es:

```
'''
[nuevo_elemento for elemento in secuencia]'''
```

- **nuevo_elemento:** Es la expresión que se evaluará y añadirá a la nueva lista.
- **elemento:** Es una variable que toma cada valor de la secuencia en cada iteración.
- **secuencia:** Es la colección de datos (como una lista, tupla, cadena, etc.) sobre la cual se itera.

Por ejemplo, supongamos que queremos crear una lista de los cuadrados de los números del 1 al 5.

```
'''
cuadrados = [i ** 2 for i in range(1, 6)]
print(cuadrados)
'''
```

Salida: [1, 4, 9, 16, 25]

Se podría haber llegado a la misma solución mediante el uso del bucle for, pero la comprensión de listas es más concisa y clara.

Se puede añadir una condición (if) dentro de la comprensión de listas para filtrar los elementos que se incluirán en la nueva lista. Si quisiésemos crear una lista con solo los números pares del 1 al 10:

```
'''
pares = [i for i in range(1, 11) if i % 2 == 0]
print(pares)
'''
```

Salida: [2, 4, 6, 8, 10]

Se pueden utilizar comprensiones de listas anidadas para trabajar con estructuras de datos más complejas, como matrices. Supongamos, por ejemplo, que queremos crear una lista que contenga los números del 1 al 3, pero repetidos 3 veces cada uno.

```
'''
repetidos = [i for i in range(1, 4) for j in range(3)]
print(repetidos)
'''
```

Salida: [1, 1, 1, 2, 2, 2, 3, 3, 3]

4- ¿Qué es un argumento en Python?

En Python, un argumento es un valor que se pasa a una función cuando se le llama a esta función. Los argumentos permiten que las funciones procesen datos y realicen operaciones con diferentes valores, sin necesidad de redefinir la función cada vez que se quiera utilizar con distintos datos. Dicho de otra manera, los argumentos son como las "entradas" de una función que permiten un comportamiento variable.

Las funciones en Python se definen con la palabra clave `def`, seguida del nombre de la función y, entre paréntesis, una lista de parámetros. Estos parámetros actúan como variables dentro de la función y reciben los valores de los argumentos cuando la función es llamada. Es decir, el argumento funcionaría como el valor que se le da a la variable (que está definida entre paréntesis) cuando se llama a la función. Por ejemplo:

```
'''
def saludar(nombre):
    print(f"Hola, {nombre}!")
'''
```

En este ejemplo, `nombre` es un parámetro de la función `saludar`. Al llamar a esta función con un argumento, dicho valor se asigna al parámetro `nombre` dentro de la función. Si se llamase a la función como `saludar("María")`, `María` sería el argumento y dará la salida

Hola, María!

Python permite trabajar con diferentes tipos de argumentos al definir y llamar a funciones:

1. Argumentos Posicionales

Estos son los argumentos más comunes y se pasan a la función en el orden en que se definen los parámetros. El primer valor se asigna al primer parámetro, el segundo valor al segundo parámetro, y así sucesivamente.

```
'''
def suma(a, b):
    return a + b

resultado = suma(3, 5)
print(resultado)
'''
```

En este caso, `3` y `5` son argumentos posicionales.

2. Argumentos por keywords o nombre

Los argumentos por nombre se pasan a la función especificando explícitamente el nombre del parámetro. Esto permite que el orden de los argumentos no sea tan estricto como en los argumentos posicionales.

```
'''
def presentar(nombre, edad):
    print(f"{nombre} tiene {edad} años.")

presentar(edad=30, nombre="Carlos")
'''
```

Aquí, se están pasando los argumentos utilizando los nombres de los parámetros, lo que hace que el orden sea irrelevante.

3. Argumentos con Valores por Defecto

Es posible definir parámetros con valores predeterminados en una función. Si no se proporciona un argumento para un parámetro con un valor por defecto, la función utilizará el valor especificado en la definición.

```
...
def saludar(nombre="amigo"):
    print(f"Hola, {nombre}!")

saludar()
saludar("Laura")
...
```

En la primera llamada, la función utiliza el valor predeterminado "amigo", mientras que en la segunda llamada, utiliza el argumento "Laura".

4. Argumentos **args* y ***kwargs*

Python también permite trabajar con un número variable de argumentos utilizando los parámetros especiales **args* y ***kwargs*.

- **args*: Se utiliza para pasar una lista de argumentos posicionales de longitud variable a una función.

```
...
def sumar_todos(*args):
    return sum(args)

resultado = sumar_todos(1, 2, 3, 4, 5)
print(resultado)
...
```

Aquí, **args* permite pasar cualquier cantidad de números a la función `sumar_todos`.

- ***kwargs*: Se utiliza para pasar una cantidad variable de argumentos por nombre a una función.

```
...
def imprimir_datos(**kwargs):
    for clave, valor in kwargs.items():
        print(f"{clave}: {valor}")

imprimir_datos(nombre="Ana", edad=28, ciudad="Madrid")
...
```

En este caso, ***kwargs* permite pasar una cantidad variable de argumentos nombrados a la función `imprimir_datos`.

5- ¿Qué es una función de Python Lambda?

En Python, una función lambda es una función sin especificar, es decir, una función que no tiene un nombre explícito y se define de manera compacta en una sola línea. Estas funciones se crean con la palabra clave `lambda`, seguida de una lista de parámetros, dos puntos (`:`) y una expresión que representa el valor de retorno de la función. Las funciones lambda son útiles cuando se necesita una función simple y corta que se utilice de manera temporal en el código.

La tipología básica de una función lambda es la siguiente:

```
'''
lambda argumentos: expresión
'''
```

- `lambda`: Es la palabra clave utilizada para definir la función.
- `argumentos`: Son los parámetros que la función tomará como entrada (puede haber uno o más).
- `expresión`: Es la operación que se realizará con los argumentos y cuyo resultado será devuelto por la función.

Aquí se muestra cómo definir una función lambda que suma dos números:

```
'''
suma = lambda x, y: x + y
'''
```

En este caso, `x` y `y` son los argumentos, y `x + y` es la expresión que se evaluará y devolverá el resultado. La función lambda se almacena en la variable `suma`, y se puede utilizar como cualquier otra función.

```
'''
resultado = suma(3, 5)
print(resultado)
'''
```

Salida: 8

Las funciones lambda constan de res características principales:

1. **Son Anónimas**: Las funciones lambda no tienen un nombre propio. Aunque pueden ser asignadas a una variable, el objetivo principal de una función lambda es realizar operaciones simples sin necesidad de una declaración.
2. **Concisas**: Las funciones lambda están diseñadas para ser compactas y se utilizan comúnmente en situaciones donde se requiere una función rápida que no se va a reutilizar en varias partes del código.
3. **Expresión Única**: A diferencia de las funciones regulares definidas con `def`, las funciones lambda solo pueden contener una única expresión. No pueden contener múltiples líneas de código o declaraciones complejas.

Aquí se muestra cómo se vería la misma funcionalidad implementada con una función lambda y con una función definida con `def`:

Con Función Lambda

```
'''
multiplicar = lambda x, y: x * y
resultado = multiplicar(3, 4)
print(resultado)
'''
```

Con Función def

```
'''
def multiplicar(x, y):
    return x * y

resultado = multiplicar(3, 4)
print(resultado)
'''
```

Ambos ejemplos producirán el mismo resultado (12), pero la función lambda es más compacta y se utiliza típicamente en situaciones donde se necesita una función pequeña y no reutilizable.

6- ¿Qué es un paquete pip?

Pip es el administrador de paquetes para Python. Se utiliza para instalar y gestionar bibliotecas y paquetes adicionales que no vienen incluidos en la instalación estándar de Python. Para instalar o desinstalar cualquier paquete disponible en el índice de paquetes de Python (PyPI) → `pip install` o `uninstall` nombre_paquete

Por ejemplo, en caso de requerir trabajar con solicitudes HTTP en Python, puedes instalar el paquete `requests` usando `pip` → `pip install requests`

Una vez instalado, puedes usar `requests` en tu código Python para realizar solicitudes HTTP.

Pip es fundamental para la gestión de dependencias en proyectos de Python, permitiendo que los desarrolladores instalen fácilmente bibliotecas necesarias para su código.